

Despliegue en AWS con Elastic Beanstalk

OE Inventory — Django 4.1 · MariaDB/PyMySQL · guía paso a paso

Guía detallada para migrar la aplicación a un servidor web de AWS usando **Elastic Beanstalk** (plataforma Python), adaptada al proyecto: secretos en **.env**, estáticos externalizados, PDFs de **staff_docs**, esquema de BBDD gestionado manualmente (**migrate --fake**), mapa Leaflet con geocodificación y recuperación de contraseña por email.

Fase 0 — Prerrequisitos

- Cuenta de AWS y usuario IAM con permisos sobre EB, EC2, RDS, S3, ACM, Route 53 y CloudWatch.
- Instalar AWS CLI y EB CLI, y configurar credenciales/región.
- Repositorio limpio y **requirements.txt** actualizado.

```
pip install awsebcli
aws configure          # Access key, Secret, región (p. ej. eu-west-1)
eb --version
```

Fase 1 — Preparar el código para producción

1.1. settings.py productivo

- **DEBUG = False**.
- **ALLOWED_HOSTS** con el dominio EB (*.elasticbeanstalk.com) y tu dominio final.
- **SECRET_KEY**, credenciales de BBDD y de email tomadas de variables de entorno (ya usas .env).
- **CSRF_TRUSTED_ORIGINS** con https://tu-dominio y https://*.elasticbeanstalk.com.
- Tras el balanceador (ALB): **SECURE_PROXY_SSL_HEADER**, **SECURE_SSL_REDIRECT**, cookies seguras y HSTS.
- Mantener **SECURE_REFERRER_POLICY** = 'strict-origin-when-cross-origin' (lo necesita el mapa).

```
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000
```

1.2. Estáticos con WhiteNoise

- Añadir 'whitenoise' a requirements y su middleware (justo tras SecurityMiddleware).
- Definir **STATIC_ROOT** y el almacenamiento comprimido de WhiteNoise.

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',
    ... ]
STATIC_ROOT = BASE_DIR / 'staticfiles'
STORAGES = {'staticfiles': {'BACKEND':
    'whitenoise.storage.CompressedManifestStaticFilesStorage'}}
```

1.3. Media (PDFs de staff) en S3

El disco local de EB **no es persistente** entre despliegues ni entre instancias, así que los PDFs subidos en staff_docs deben ir a S3 (ver Fase 3). Usar boto3 + django-storages.

1.4. requirements.txt

- Añadir: gunicorn, whitenoise, boto3, django-storages.
- Valorar mysqlclient en lugar de PyMySQL para producción.

1.5. Ficheros de Elastic Beanstalk

Procfile (arranque con gunicorn) y .ebextensions para collectstatic/migrate. Recuerda que el proyecto vive en **src/**, así que el WSGIPath y los comandos deben apuntar allí.

```
# Procfile
web: gunicorn --chdir src oe_inventory.wsgi:application

# .ebextensions/01_django.config
container_commands:
  01_collectstatic:
    command: "cd src && python manage.py collectstatic --noinput"
  02_migrate:
    command: "cd src && python manage.py migrate --fake-initial"
    leader_only: true
```

1.6. Secretos

- NO subir .env al repositorio (verifica .gitignore).
- Las variables van como Environment Properties de EB, o en SSM Parameter Store / Secrets Manager.

Fase 2 — Base de datos (RDS for MariaDB)

- Crear instancia RDS MariaDB (versión compatible con Django 4.1; revisa la rama 10.x) en VPC privada.
- Migrar datos: mysqldump de la BBDD actual y restaurar en RDS (conserva esquema y datos).
- Tras restaurar: **manage.py migrate --fake** para alinear el estado de migraciones con el esquema real.
- Security Groups: la BBDD solo accesible desde el SG de la app (nunca pública).
- Pasar host/port/name/user/password como Environment Properties.

```
mysqldump -h ORIGEN -u USER -p oees_db > oees_db.sql
mysql -h ENDPOINT_RDS -u USER -p oees_db < oees_db.sql
cd src && python manage.py migrate --fake
```

IMPORTANTE: el esquema tiene columnas gestionadas a mano. NO crees el esquema desde cero con migraciones; restaura el dump y usa --fake.

Fase 3 — Media en S3

- Crear bucket S3 (con la política de acceso adecuada).
- Usuario/rol IAM con permisos sobre el bucket (credenciales por env o rol de instancia).
- Configurar django-storages para que staff_docs use S3 como almacenamiento por defecto.
- (Opcional) Subir al bucket los PDFs ya existentes.

Fase 4 — Crear la aplicación y el entorno EB

- eb init (región, plataforma Python 3.x, nombre de la app).
- eb create (entorno con load balancer ALB y tipo de instancia).
- Definir Environment Properties con TODAS las variables del .env (SECRET_KEY, DB_*, EMAIL_*, AWS_*...).
- Usar la RDS externa de la Fase 2 (preferible a una acoplada al entorno, que se borra con él).

```
eb init -p python-3.11 oe-inventory
eb create oe-inventory-prod --elb-type application
```

Fase 5 — Desplegar

- eb deploy.
- Verificar que en el deploy se ejecutan collectstatic y migrate (--fake donde toque).
- Crear superusuario si hace falta (eb ssh -> manage.py createsuperuser) o usar uno migrado.
- eb open y comprobar que la aplicación carga.

Fase 6 — HTTPS y dominio

- Solicitar certificado en ACM para tu dominio.
- Configurar el ALB del entorno para HTTPS (443) con ese certificado y redirigir 80 -> 443.
- Route 53: registro DNS apuntando al ALB del entorno.
- Revisar ALLOWED_HOSTS y CSRF_TRUSTED_ORIGINS con el dominio definitivo.

Fase 7 — Email (recuperación de contraseña)

- Configurar EMAIL_HOST_USER / EMAIL_HOST_PASSWORD / DEFAULT_FROM_EMAIL como variables de entorno.
- Asegurar salida a Internet desde las instancias (NAT Gateway si están en subred privada): lo necesitan el envío de email, la geocodificación Nominatim y, en el navegador, las teselas del mapa (CARTO).
- Probar el flujo 'Forgot my password' de extremo a extremo.

Fase 8 — Verificación y operación

- Smoke test de cada formulario; probar subida de PDF (S3), mapa de delegaciones, exportaciones Excel y perfiles reader.
- Logs a CloudWatch (ya tienes LOGGING); revisar con eb logs.
- Backups automáticos de RDS y snapshots.
- Proceso de redeploy (eb deploy) y rollback.
- (Opcional) Escalado y health checks del entorno.

Puntos críticos específicos de esta app

- **migrate --fake** tras restaurar la BBDD (columnas gestionadas a mano).
- **PDFs a S3**: el disco de EB no persiste.
- **Salida a Internet** de las instancias para email y geocodificación.
- **ALLOWED_HOSTS** y **CSRF_TRUSTED_ORIGINS** con el dominio EB/definitivo.
- **SECURE_REFERRER_POLICY** 'strict-origin-when-cross-origin' (teselas del mapa).

Documento generado para OE Inventory.